# Precise scheduler for FreeRTOS

## Cybersecurity Engineering MSc - OSES Course

Group 7

Cybersecurity Engineering MSc

February 18, 2026

Politecnico di Torino

# Task Creation API

- Task create functions inspired by FreeRTOS `xTaskCreate` API:

```
BaseType_t xTaskHrtCreate(
    TaskFunction_t pxTaskCode,
    const char* pcName,
    void* pvParameters,
    uint32_t ulSubframeId,
    uint32_t ulStartTime,
    uint32_t ulEndTime,
    HRTHandle_t* pxTaskReturn);

BaseType_t xTaskSrtCreate(
    TaskFunction_t pxTaskCode,
    const char* pcName,
    void* pvParameters,
    uint32_t ulSubframeId,
    SRTHandle_t* pxTaskReturn);
```

- Design choice to provide a clean API and validate arguments:
  - i.e.: check subframe exists and prevent HRT task overlap
- SRT order of execution in subframe depends on order of creation

# Task Creation API (static)

- Support for static create variants (user-provided TCB memory)

```
HRTHandle_t xTaskHrtCreateStatic(
    TaskFunction_t pxTaskCode,
    const char* const pcName,
    void* pvParameters,
    uint32_t ulSubframeId,
    uint32_t ulStartTime,
    uint32_t ulEndTime,
    HRTTask_t* const pxTaskBuffer);

SRTHandle_t xTaskSrtCreateStatic(
    TaskFunction_t pxTaskCode,
    const char* const pcName,
    void* pvParameters,
    uint32_t ulSubframeId,
    SRTTask_t* const pxTaskBuffer);
```

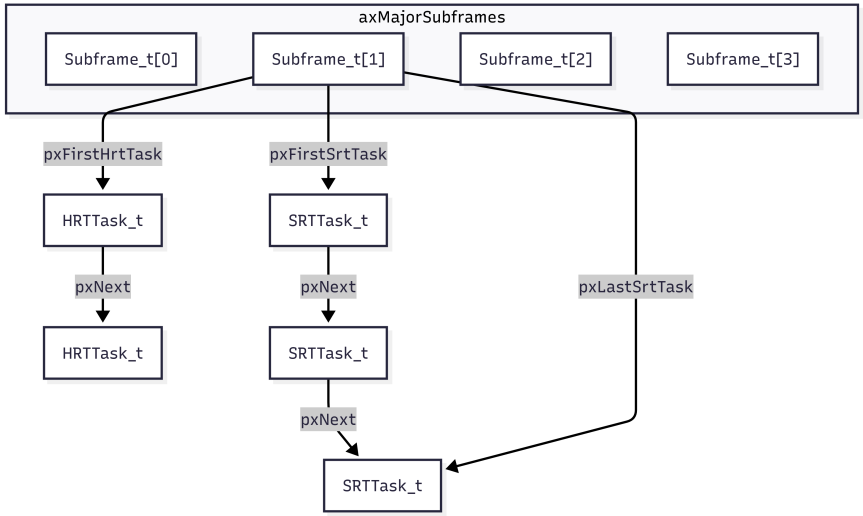
- Design choice to permit compiling without dynamic allocation
- Static or regular API must be called before `vStartScheduler`
- Function in `pxTaskCode` can safely return when job is finished

# Task Data Structures

- Internal scheudler structures managed safely through external APIs

```
static Subframe_t axMajorSubframes[configSUBFRAME_COUNT_PER_MAJOR];

typedef struct xHRT_TASK {
    TCB_t xTCB; // or StaticTask_t
    TaskFunction_t pxTaskCode;
    void* pvParameters;
    uint32_t ulStartTime;
    uint32_t ulEndTime;
    struct xHRT_TASK* pxNext;
} HRTTask_t;

typedef struct xSUBFRAME {
#if configENABLE_HRT_SCHEDULER
    HRTTask_t* pxFirstHrtTask;
    uint32_t ulHrtCount;
#endif
#if configENABLE_SRT_SCHEDULER
    SRTTask_t* pxFirstSrtTask;
    SRTTask_t* pxLastSrtTask;
    uint32_t ulSrtCount;
#endif
} Subframe_t;

typedef const struct xHRT_TASK* HRTHandle_t;
typedef const struct xSRT_TASK* SRTHandle_t;
```

```
typedef struct xSRT_TASK {
    TCB_t xTCB; // or StaticTask_t
    TaskFunction_t pxTaskCode;
    void* pvParameters;
    struct xSRT_TASK* pxNext;
} SRTTask_t;
```




- Safe task handles, compatible with original FreeRTOS task pointer

# Task Stack Management

- HRT/SRT tasks reuse stack across same task type to save memory
- Stack provisioning is similar to `vApplicationGetIdleTaskMemory`
- Also when dynamic allocation is active (invoked at scheduler start):

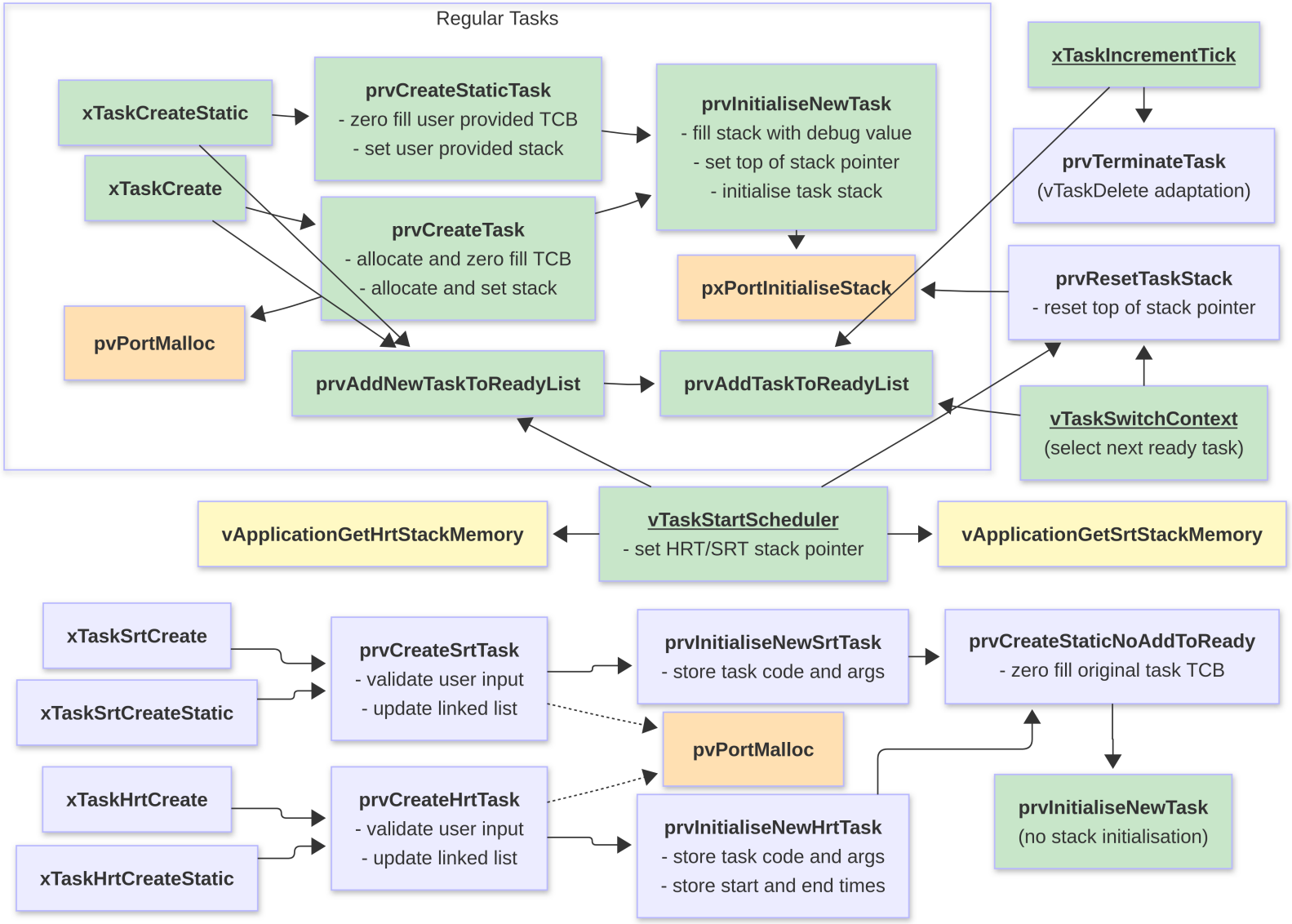
```
void vApplicationGetHrtStackMemory(
    StackType_t **ppxStackBuffer, uint32_t *pulStackSize
) {
    static StackType_t myHrtStack[128];
    *ppxStackBuffer = myHrtStack;
    *pulStackSize = 128;
}

void vApplicationGetSrtStackMemory(
    StackType_t **ppxStackBuffer, uint32_t *pulStackSize
) {
    static StackType_t mySrtStack[256];
    *ppxStackBuffer = mySrtStack;
    *pulStackSize = 256;
}
```

- User defined, unless `configKERNEL_PROVIDED_STATIC_MEMORY`

# Configuration Macros

- Realtime scheduler components can be used independently:
  - `configENABLE_HRT_SCHEDULER`, `configENABLE_SRT_SCHEDULER`
- Selection of subframe duration and count:
  - `configSUBFRAME_DURATION` (in milliseconds)
  - `configSUBFRAME_COUNT_PER_MAJOR`
- Referenced original FreeRTOS configurations:
  - `configSUPPORT_STATIC_ALLOCATION`
  - `configSUPPORT_DYNAMIC_ALLOCATION`
  - `configKERNEL_PROVIDED_STATIC_MEMORY`, when enabled uses:
    - `configHRT_TASKS_STACK_SIZE`, `configSRT_TASKS_STACK_SIZE`
- Define task priority as best suitable:
  - `configHRT_TASKS_PRIORITY`, `configSRT_TASKS_PRIORITY`
  - Realtime tasks do not break multilevel priority based scheduling!
- Testing, unsafe for production:
  - `INCLUDE_vPrintSubframes`
  - `INCLUDE_vPrintSubframesJson`

# Implementation (Call Graph)



# HRT vs SRT Scheduling

## HRT

### vTaskStartScheduler

Set `pxWaitingHRT` to subframe 0 first HRT. If task exists and has start time = 0:

- set `pxCurrentHRT = pxWaitingHRT`
- advance `pxWaitingHRT` to next task
- reset task stack, add it to ready list

### xTaskIncrementTick

Preempt `pxCurrentHRT` on deadline miss. Set `pxWaitingHRT` to the first subframe HRT when a new subframe is entered. When waiting task start time is reached:

- set `pxCurrentHRT = pxWaitingHRT`
- advance `pxWaitingHRT` to next task
- mark task to reset, add to ready list

### vTaskSwitchContext

Regularly select next task to run. If it is the HRT marked to be reset, unmark it and reset task stack before running it (critical section).

## SRT

### vTaskStartScheduler

Set `pxCurrentSRT` to subframe 0 first SRT. If such task exists:

- reset task stack, add it to ready list

### xTaskIncrementTick

At subframe start: possibly terminate current SRT if running from previous subframe; set `pxCurrentSRT` to first SRT of subframe, add it to ready list, and mark it to be reset.

### vTaskSwitchContext

If `pxCurrentSRT` terminated: advance it to next, mark to be reset and add to ready list. Regularly select next task to run. If it is the SRT marked to be reset, unmark it and reset task stack before running it (critical section).

*Marking HRT/SRT to be reset:* pointing its TCB in `pxResetHRT`/`pxResetSRT` variable.